

AZIEL RUNTIME

Adaptive Knowledge Recollection, Bayesian Calibration & 3-of-4 Triad Selection

Implementation Whitepaper for Grok Bot - v1.0

Author / system owner: Aziel Eliab

Date: 2026

PRIMARY OBJECTIVE: Extend the existing ChainLock learn/recall substrate into an adaptive AI knowledge memory that uses only the calibration signals relevant to the current use case, updates confidence with Bayesian probability, selects the best 3 of 4 calibration channels, emits a Triad Score, and NEVER rewrites accepted history.

1. Executive Summary

ChainLock already provides the essential substrate: append-only hashed stamps, dedicated session/acts/evidence/recall/library/learn chains, bounded recall depth, a learn path, and hash-grounded cards returned to the AI. The missing layer is adaptive calibration: the system currently remembers and recalls, but it does not yet systematically measure which memories, heuristics, patterns, or predictions become more or less reliable as outcomes accumulate.

This whitepaper defines the implementation required to turn that substrate into adaptive knowledge recollection without violating the forward-only architecture. The system must preserve every prior observation, infer confidence from accumulated evidence rather than delete old beliefs, and expose why a memory was selected and how its probability changed.

The adaptive layer has four calibration channels. For each use case, a deterministic selector chooses the three most relevant channels. Those three produce a Triad Score. Bayesian posterior probability is retained separately and may also participate as one of the selected channels. This prevents one universal scoring formula from being misapplied to every domain.

2. Existing Substrate - Do Not Rebuild What Already Exists

Existing component	Current behavior	Adaptive extension
ChainLock append	Append-only JSONL stamp chain with prev hash and fact hash.	Preserve unchanged as evidence/history authority.
learn chain	choose/refuse/learn interactions can be copied into learn.	Add structured calibration metadata to learn events.
recall()	Searches hash-grounded facts across session/acts/recall/learn with bounded depth.	Rank results adaptively instead of returning only recency/query matches.
groundedDoor()	Requires stamped facts before returning grounded context.	Return calibrated memory cards with posterior + Triad metadata.
Worker storage	In-memory or USES KV; local CLI vault remains documented source.	Derived indexes may be rebuilt; do not make them historical authority.
RoseClock / TemporalLock links	ChainLock stamps can carry rose_transition_hash and temporal_hash.	All adaptive updates must carry forward-state references.

Rule: ChainLock remains the immutable recollection ledger. The adaptive memory index is derived state. If the index is lost, it must be reconstructible from hashed events.

3. Constitutional Invariants

- History never changes. New evidence appends a new event; it does not modify or delete the earlier belief.
- Bayesian probability is calibrated belief strength, not truth.
- Retrieval weight may change. Historical evidence may not.
- Corrections, supersessions, revocations, and restore-forward operations are new RoseClock transitions.
- No AI component self-grants MODEL_UPDATE, TRUST_UPDATE, NETWORK_EGRESS, MESSAGE_SEND, or LOCAL_EXEC authority.
- Every adaptive update records the evidence used, prior state hash, posterior state hash, and calibration version.
- Every selected memory returned to an AI can be traced back to one or more ChainLock hashes.
- If calibration inputs are missing, the system degrades gracefully and records uncertainty rather than inventing certainty.
- Use-case selection chooses the relevant 3 of 4 calibration channels; it does not cherry-pick whichever three produce the highest score.

4. Four Calibration Channels

The adaptive system uses four orthogonal calibration channels. The four are intentionally broad enough to work across the 33-software architecture while accepting engine-specific inputs.

Code	Question answered	Example inputs
E - Evidence / Provenance Strength	How complete, direct, independently sourced, replicated, and provenance-grounded is the underlying evidence?	SPRE evidence completeness; source trust; corroboration; replication; chain-of-custody.
C - Cross-Layer Consistency	How well do representation, description, and observed/functional reality align?	AZ-CLCE $R \leftrightarrow D$, $D \leftrightarrow P$, $R \leftrightarrow P$, negative-space N, contradiction rate.
P - Pattern / Structural Fit	How well does the item match a validated structural pattern or domain ontology without converting similarity into guilt/truth?	SPRE SSI/Pattern Confidence; ZionPattern ontology; domain-specific pattern score.
B - Bayesian Outcome Calibration	Given prior resolved outcomes, how well has this memory/model/pattern family actually performed?	Beta posterior, effective sample size, Brier score, calibration error.

4.1 Evidence channel E

$E = \text{weighted_mean}(\text{completeness}, \text{provenance}, \text{independence}, \text{replication}, \text{source_quality}, \text{chain_integrity})$

SPRE explicitly separates structural similarity from evidence strength: its Pattern Confidence multiplies structural score by evidence score. It also requires negative controls and threshold calibration. Preserve that distinction when SPRE feeds the adaptive memory.

4.2 Consistency channel C

$C = f(R_D, D_P, R_P, \text{negative_space_penalty}, \text{contradiction_penalty})$

AZ-CLCE defines consistency across Representation, Description, and Reality and extends the score with missing expected elements N. Use CLCE as a consistency feature, not an intent detector. Its own whitepaper states that inconsistency does not establish intent.

4.3 Pattern channel P

$P = \text{validated_pattern_fit} * \text{evidence_applicability} * \text{domain_transfer_penalty}$

Pattern scores are permitted only where a domain has a defined ontology or validated comparison set. ZionPattern demonstrates the correct restraint: explicit uncertainty, falsifiability, receipts, and a hard confidence ceiling in that prototype. The adaptive Runtime does not need a universal 75% cap, but domain caps must be honored when the originating engine defines them.

4.4 Bayesian channel B

```
Prior: Beta(alpha0, beta0)
For outcome i with evidence weight w_i and outcome x_i in [0,1]:
  alpha <- alpha + w_i*x_i
  beta  <- beta  + w_i*(1-x_i)
Posterior mean B = alpha / (alpha + beta)
```

Fractional outcomes permit graded success, partial corroboration, or domain-specific correctness measures. The evidence weight must be bounded so one event cannot overwhelm the history.

5. AI 3-of-4 Triad Selector

New requirement: for each use case, the AI selects exactly THREE of the FOUR calibration channels {E, C, P, B}, then emits both the selected triad and a Triad Score.

Selection must be explainable and primarily deterministic. An LLM may classify the use case, but the Runtime should map that classification to channel suitability rules stored in a versioned manifest. This prevents score-shopping.

5.1 Suitability scoring

```
For each channel j in {E,C,P,B}:
  suitability_j =
    0.35 * domain_manifest_fit
    + 0.25 * input_availability
    + 0.20 * historical_sample_adequacy
    + 0.20 * decision_relevance
```

Select the top 3 suitability_j values.

Tie-break order is declared by domain manifest, never by resulting confidence.

If fewer than three channels have minimally adequate data, return TRIAD_INCOMPLETE rather than fabricate a full score.

5.2 Recommended use-case defaults

Use case	Default triad	Reason
Forensic / evidence review	E + C + P	Evidence strength, contradiction consistency, and structural pattern fit matter most; outcome history may be sparse.
Repeated prediction / operational decision	E + C + B	Evidence and consistency matter, but historical calibration becomes more useful than generic pattern fit.
Known pattern-family classification	E + P + B	Pattern fit plus actual historical performance; cross-layer consistency optional if R/D/P layers are not meaningful.
System / documentation validation	E + C + B	CLCE alignment plus observed resolution history.
Early exploratory research	E + C + P	Avoid over-weighting Bayesian history when effective sample size is too small.

AZIEL RUNTIME - ADAPTIVE KNOWLEDGE MEMORY / TRIAD CALIBRATION

Mature repeated workflow	C + P + B	Permitted only when provenance/evidence completeness is already enforced upstream by schema and remains above its domain floor.
--------------------------	-----------	---

5.3 Triad Score

The score should reward high average quality while penalizing an imbalanced triad in which one leg is weak.

Use normalized inputs in [0,1].

```
Given selected scores s1,s2,s3 and manifest weights w1,w2,w3 (sum=1):
  base = w1*s1 + w2*s2 + w3*s3
  balance = 1 - normalized_stddev(s1,s2,s3)
  completeness = observed_required_inputs / required_inputs
```

```
TRIAD_SCORE = 100 * base * (0.80 + 0.20*balance) * completeness
```

Return both raw legs and final score. Never return only the composite.

This is a proposed implementation inspired by the user-specified 3-of-4 behavior. The exact public-library Triad implementation could not be located in the accessible repository source during this build, so Grok should not label this formula as identical to the site unless it verifies the live implementation first.

5.4 Triad decision object

```
TriadDecision {
  version: "TRIAD-0.1",
  use_case,
  candidates: { E, C, P, B },
  suitability: { E, C, P, B },
  selected: ["E","C","B"],
  omitted: "P",
  omission_reason,
  component_scores: { E, C, B },
  weights: { E, C, B },
  balance,
  completeness,
  triad_score,
  confidence_class: "LOW"|"MODERATE"|"HIGH",
  evidence_refs: [],
  chain_refs: [],
  policy_version
}
```

6. Adaptive Memory Data Model

```
AdaptiveMemoryNode {
  memory_id,
  subject_key,
  relation_key?,
  object_key?,
  source_chain_hashes[],
  provenance_refs[],
  first_seen_rose_hash,
  latest_state_rose_hash,

  status: ACTIVE|CONTESTED|SUPERSEDED|REVOKED,
  supersedes?,
  contradicted_by[],
  supported_by[],

  alpha,
  beta,
  posterior_probability,
  posterior_variance,
  effective_observations,
  brier_score?,
```

AZIEL RUNTIME - ADAPTIVE KNOWLEDGE MEMORY / TRIAD CALIBRATION

```
channel_scores: { E?, C?, P?, B? },
latest_triad?,
retrieval_weight,
retrieval_version,
last_recalled_at?,

index_hash
}
```

AdaptiveMemoryNode is a DERIVED index object. It must never replace the underlying ChainLock events. Rebuildability is mandatory.

7. Event Types to Add to ChainLock Learn

Event	Meaning
memory_observation	A new candidate fact, relation, claim, pattern, or experience is observed.
memory_support	New evidence supports an existing memory.
memory_contradiction	New evidence conflicts with an existing memory.
memory_resolution	A prediction/claim receives a resolved or graded outcome.
calibration_update	Bayesian posterior and Triad data are recomputed from immutable events.
memory_supersede	A newer interpretation becomes preferred while old memory remains addressable.
memory_revoke	Authority withdraws operational use; history remains.
retrieval_feedback	Records whether recalled context proved useful/relevant, without equating usefulness with truth.

8. Bayesian Calibration Details

8.1 Priors

Use weak, explicit priors unless a domain manifest declares a justified prior. Default Beta(1,1) is uniform. Do not encode developer intuition as an invisible strong prior.

DEFAULT prior: alpha=1, beta=1

8.2 Evidence-weighted update

```
quality = mean(available normalized quality signals)
w = clamp(0.05, 4.0, 0.5 + 1.5*quality)
x = normalized observed outcome [0,1]
alpha_new = alpha_old + w*x
beta_new = beta_old + w*(1-x)
```

8.3 Calibration quality

Brier = mean((predicted_probability - observed_outcome)^2)

Track Brier score by memory family, engine, operation, and optionally domain. Lower is better. Do not merge materially different task families merely to increase sample size.

8.4 Small-sample restraint

Expose effective sample size. A posterior of 0.90 from 2 effective observations is not equivalent to 0.90 from 2,000. Retrieval and decision logic should receive both probability and effective N.

9. Adaptive Retrieval

Current recall is primarily chain/depth/query based. Upgrade it to ranked retrieval using lexical relevance, Bayesian calibration, evidence strength, recency, and status.

```
retrieval_score =
  0.35 * semantic_or_lexical_relevance
+ 0.25 * posterior_reliability
+ 0.20 * evidence_strength
+ 0.10 * recency
+ 0.10 * use_case_triad_fit

status multipliers:
ACTIVE      1.00
CONTESTED   0.75
SUPERSEDED  0.35 (unless query asks history)
REVOKED     0.00 for operational use; still retrievable for audit/history
```

Weights should live in a versioned calibration manifest and may be adapted only through an authorized forward learning update.

10. What "Uses What It Needs to Calibrate" Means

The system must not run every analytical engine for every memory. It should inspect the use case and available evidence, then request only useful calibration features.

```
Use-case router
-> inspect domain + operation + input packet
-> determine available calibration channels
-> score E/C/P/B suitability
-> choose exactly 3
-> call only the feature providers needed for those 3
-> compute Triad Score
-> update Bayesian posterior if an outcome exists
-> append calibration_update to ChainLock learn
-> rebuild/update derived memory index
```

Examples: CLCE is called when R/D/P consistency is meaningful. SPRE is called when suppression-pattern structural similarity is relevant. ZionPattern is called when its cold-case ontology is relevant. Generic research does not invoke them merely because they exist.

11. Domain Engine Feature Adapter Contract

```
CalibrationFeatureProvider {
  provider_id,
  version,
  domains[],
  channels_supported: ["E","C","P"],
  can_score(input_packet, use_case) -> suitability [0,1],
  score(input_packet) -> {
    channel,
    score [0,1],
    evidence_refs[],
    limitations[],
    uncertainty [0,1],
    provider_receipt_hash
  }
}
```

Provider outputs are evidence to the calibrator, not authority over the final memory state.

12. Grok Bot Implementation Plan

1. Create src/chainlock/adaptive.js for Bayesian math, evidence weighting, posterior reconstruction, retrieval scoring, and Triad scoring.
2. Extend ChainLock stamp schema to accept a bounded memory metadata object. Ensure hashFieldsOf() includes it so calibration metadata is integrity-protected.
3. Add event kinds listed in Section 7 to the learn chain; never mutate old rows.
4. Create src/memory/index.js: rebuild derived AdaptiveMemoryNode objects by replaying learn-chain events.
5. Create src/memory/triad.js: deterministic channel suitability, 3-of-4 selection, Triad Score, and explicit omitted-channel reason.
6. Create src/memory/providers/: clce.js, spre.js, zionpattern.js, generic-evidence.js, bayes.js. Providers must be optional and domain-gated.
7. Add a versioned calibration manifest under src/memory/calibration-manifest.js or JSON.
8. Upgrade recall() or add adaptiveRecall() so normal ChainLock verification remains untouched and adaptive ranking is an additive path.
9. Feed adaptive recalled cards to AZAI/AZBot only after ChainLock verification succeeds.
10. Wire resolved outcomes into memory_resolution events. Do not infer outcomes merely from model confidence.
11. Add Oracle-style compare/update loop only after outcome evidence is present and authorized.
12. For every accepted calibration update: RoseClock forward transition -> TemporalLock evidence -> ChainLock learn stamp.
13. Expose read-only diagnostic endpoints/tools for memory explainability: posterior, selected triad, component legs, evidence refs, effective N, Brier score.
14. Add migration that can rebuild the index from existing learn stamps without changing existing ChainLock hashes.
15. Add unit/property/security tests before enabling automatic model updates.

13. Suggested API

```
POST /v1/memory/observe
POST /v1/memory/resolve
POST /v1/memory/calibrate
POST /v1/memory/recall
GET /v1/memory/{id}
GET /v1/memory/{id}/history
GET /v1/memory/{id}/calibration
POST /v1/memory/rebuild-index // OPERATOR / local only
```

These endpoints must remain behind FragGate. Lamb Lens and the normal security chain still apply to executable operations.

14. Example: Adaptive Calibration Cycle

1. User / engine observes claim H.
2. ChainLock appends memory_observation(H).
3. Use-case classifier = repeated operational decision.
4. Suitability selects E + C + B; P omitted.
5. Evidence provider scores E=.82.
6. CLCE adapter scores C=.74.
7. Prior Beta(1,1); historical outcomes yield B=.68, effective N=14.3.
8. Triad(E,C,B) = computed composite, with all three legs retained.
9. AI recalls H with calibrated weight; it does not treat H as proven.
10. Later observed outcome contradicts H with strong evidence.

AZIEL RUNTIME - ADAPTIVE KNOWLEDGE MEMORY / TRIAD CALIBRATION

11. Append `memory_resolution(outcome=0, evidence_weight=1.7)`.
12. Recompute posterior -> B falls.
13. Append `calibration_update` with old/new posterior hashes.
14. RoseClock advances. Old belief and old posterior remain auditable.
15. Future recall naturally gives H less operational weight.

15. Example: Forensic / Historical Research

For a disputed historical case, the likely default triad is E + C + P. SPRE can contribute E and P because it explicitly separates structural suppression similarity from evidence completeness and frames outputs as triage rather than guilt. CLCE contributes C by measuring R/D/P consistency and negative space. Bayesian B becomes more useful only after enough resolved comparable cases exist.

Selected: E + C + P
Omitted: B (insufficient comparable resolved outcomes)
Result: Triad Score + explicit E/C/P legs + uncertainty + source hashes

16. Anti-Apophenia / Anti-Self-Reinforcement Controls

- Never count an AI-generated conclusion as independent corroboration of itself.
- Deduplicate evidence by content/provenance hash before weighting corroboration.
- Separate source independence from source count.
- Negative controls are mandatory for pattern engines used in calibration.
- Contradictions lower operational weight but remain retrievable.
- Outcome labels require evidence provenance and must distinguish UNKNOWN from MISS.
- Do not allow repeated recalls to increase truth probability; retrieval frequency is not evidence.
- Do not allow a high Triad Score to automatically authorize an action.
- Calibrators cannot bypass Lamb Lens, DecisionGATE, domain capability checks, or RoseClock.
- Preserve domain-specific confidence caps such as ZionPattern 75% where applicable.

17. Tests Grok Must Add

Test	Required pass condition
Append-only	Existing learn rows and their hashes are byte-for-byte unchanged after calibration.
Posterior reconstruction	Replaying immutable resolution events reproduces the same alpha/beta/posterior.
Evidence weighting	Low-quality evidence moves posterior less than high-quality evidence.
Unknown outcome	UNKNOWN does not count as failure or success.
Triad exactly 3	Selector returns three unique channels or TRIAD_INCOMPLETE.
No score shopping	Changing component scores without changing suitability cannot alter which channels are selected.
Provider optionality	Unrelated use case does not invoke SPRE/CLCE/ZionPattern unnecessarily.
Small N	Effective N appears and prevents high posterior from being represented as mature calibration.
Brier	Known synthetic probabilities produce expected Brier score.
Supersession	Superseding creates a new event; old memory remains queryable historically.
Tamper detection	Altered memory metadata breaks stamp verification.
Rebuild	Delete derived index; rebuild from ChainLock produces identical index hash.
Concurrency	Two calibration writes against same RoseClock tip cannot both

AZIEL RUNTIME - ADAPTIVE KNOWLEDGE MEMORY / TRIAD CALIBRATION

	silently commit.
Capability	MODEL_UPDATE required for adaptive model changes; recall alone cannot update.
Privacy	Secrets/raw sensitive payloads are not copied into memory metadata or receipts.

18. Minimal Pseudocode

```

async function adaptiveLearn(ctx) {
  const verified = await ChainLock.verify(ctx.env, { chain: "learn" });
  if (!verified.ok) throw new Error("CHAIN_VERIFY_FAIL");

  const useCase = classifyUseCase(ctx.packet, ctx.operation);
  const availability = await inspectCalibrationAvailability(ctx, useCase);
  const triad = selectThreeOfFour(useCase, availability, CALIBRATION_MANIFEST);
  if (!triad.ok) return { state: "TRIAD_INCOMPLETE", triad };

  const channelScores = await scoreSelectedChannels(triad.selected, ctx);
  const posterior = rebuildPosterior(ctx.learnEvents, ctx.prior);

  // B is computed from outcome history; providers must not overwrite it.
  channelScores.B = posterior.probability;

  const scored = computeTriadScore(triad, channelScores);
  const delta = buildCalibrationDelta(ctx, posterior, scored);

  const rose = await RoseClock.commitForward({
    expected_parent: ctx.expected_rose_tip,
    action_class: "LEARN",
    delta_hash: hash(delta),
    authority: ctx.capabilities
  });

  const temporal = await TemporalLock.append({ rose });
  const stamp = await ChainLock.append(ctx.env, {
    c: "learn",
    k: "calibration_update",
    subject: ctx.subject,
    fact: summarize(delta),
    memory: delta,
    rose_transition_hash: rose.hash,
    temporal_hash: temporal.hash,
    provenance_hash: ctx.packet.content_hash
  });

  await DerivedMemoryIndex.apply(stamp);
  return { posterior, triad: scored, stamp: stamp.card };
}

```

19. Memory Recall Pseudocode

```

async function adaptiveRecall(env, query, useCase) {
  const raw = await ChainLock.recall(env, { query, depth: 5 });
  if (!raw.ok) return raw;

  const ranked = [];
  for (const card of raw.facts) {
    const node = await MemoryIndex.byChainHash(card.h);
    const score = retrievalScore({
      relevance: relevance(query, card),
      posterior: node?.posterior_probability ?? 0.5,
      evidence: node?.channel_scores?.E ?? 0.5,
      recency: recency(card.t),
      triad_fit: triadFit(useCase, node?.latest_triad)
    });
  }
}

```

```

        AZIEL RUNTIME - ADAPTIVE KNOWLEDGE MEMORY / TRIAD CALIBRATION
        ranked.push({ card, node, score });
    }

    return ranked.sort((a,b) => b.score-a.score).slice(0, MEMORY_CONTEXT_CAP);
}

```

20. Relationship to the Uploaded Whitepapers

SPRE provides a suitable model for separating pattern similarity from evidence completeness, using negative controls, blinded scoring, threshold calibration, and non-accusatory interpretation. The adaptive memory should consume those outputs as bounded features rather than elevate SPRE to a universal truth engine.

AZ-CLCE provides a consistency feature across representation, description, and reality, plus a negative-space penalty for missing expected elements. Its stated limitation - inconsistency does not establish intent - should be preserved in all memory metadata.

ZionPattern provides strong design precedents for explicit uncertainty, falsifiability, immutable receipts, and non-adaptive failure avoidance. Its hard 75% cap is domain-specific and should remain binding when results originate in ZionPattern rather than being generalized to every Runtime probability.

21. Implementation Definition of Done

- ChainLock learns and recalls as before; existing verification passes.
- New calibration events are fully hash-protected.
- Bayesian posterior can be deterministically rebuilt from event history.
- Adaptive recall ranks memories using current calibrated reliability without deleting historical context.
- AI selects exactly 3 of 4 calibration channels based on use-case suitability, not score maximization.
- Triad Score exposes its three component legs, omitted leg, omission reason, completeness, and version.
- SPRE/CLCE/ZionPattern adapters are only invoked for relevant use cases.
- Brier score and effective sample size are tracked.
- Oracle-style updates are forward-only RoseClock transitions.
- No rollback, no history rewrite, no self-trust escalation, no hidden confidence inflation.
- Full unit/property/security test suite passes.
- Documentation clearly distinguishes belief calibration, pattern similarity, and truth.

22. Grok Bot Build Instruction - Copyable

Implement this specification as an additive upgrade to aziel-runtime. Preserve the current ChainLock hash format unless schema versioning is required. Do not mutate any accepted chain history. Build adaptive memory by replaying immutable learn events, add Bayesian outcome calibration, add deterministic 3-of-4 Triad selection, and add explainable adaptive recall. Use SPRE, AZ-CLCE, ZionPattern, and other engines only when the use-case manifest says they are relevant. Every calibration update must be forward-only, provenance-grounded, capability-bound, reconstructible, and receipt-verifiable.

Appendix A - Source Basis

Source	Design input used
Implementation whitepaper v1.0 Forward-only, Bayesian-calibrated, hash-grounded	

AZIEL RUNTIME - ADAPTIVE KNOWLEDGE MEMORY / TRIAD CALIBRATION

Current ChainLock source (aziel-runtime/src/chainlock/ops.js)	Append-only stamp chains; roster includes recall and learn; recall searches session/acts/recall/learn; interact writes choose/refuse/learn events to learn; groundedDoor returns stamped facts.
Current ChainLock storage (aziel-runtime/src/chainlock/store.js)	Worker in-memory or USES KV under chainlock prefix; local vault JSONL path documented.
SPRE Structural Suppression Engine	Formal structural-similarity framework; evidence completeness; Pattern Confidence = structural score times evidence; negative controls; threshold calibration; non-accusatory interpretation.
AZ-CLCE Whitepaper v1/v2	R/D/P consistency model, mismatch scan, negative-space N, explicit limitation that inconsistency does not establish intent.
ZionPattern Solver v0.2	75% domain confidence cap, 25% uncertainty floor, immutable receipts, falsifiability, pattern evolution from unresolved gaps.
Aziel Runtime 1.7.0 source	MASTER-33 execution order: FragGate -> Lamb Lens -> SweepGate -> Sentinel -> Provenance -> ChainLock-IN -> DecisionGATE -> AZPIPE -> Internal Domain -> RoseClock -> TemporalLock -> ChainLock-OUT -> ForgeReceipts.

Appendix B - Non-goals

- No claim that Bayesian posterior equals objective truth.
- No autonomous accusation from SPRE/CLCE/pattern scores.
- No universal 75% cap outside engines that define one.
- No memory reinforcement from repetition alone.
- No hidden deletion of superseded memories.
- No requirement to run all four calibration channels on every use case.
- No direct model-weight training unless separately authorized and implemented under MODEL_UPDATE.